

Praca Dyplomowa Inżynierska

Adam Kliś
217584

Projekt systemu autoryzacji w oparciu o funkcjonalność tokenów

Design of an authorization system based on token functionality

Informatyka

Praca wykonana pod kierunkiem
dr inż. Artur Krupa
Katedra Informatyki / Instytut Informatyki Technicznej

Warszawa, rok 2026



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Słownik pojęć

- **Autoryzacja** – proces weryfikacji, czy dany (już uwierzytelniony) użytkownik posiada uprawnienia do wykonania określonej operacji.
- **Uwierzytelnianie (Authentication)** – proces weryfikacji tożsamości użytkownika (np. na podstawie loginu i hasła).
- **Token** – cyfrowy nośnik informacji, często zabezpieczony kryptograficznie, reprezentujący tożsamość lub uprawnienia użytkownika.
- **API (Application Programming Interface)** – interfejs programistyczny aplikacji pozwalający na komunikację między systemami.
- **Zero Trust** – model bezpieczeństwa zakładający, że żadne urządzenie ani użytkownik nie jest domyślnie zaufany, nawet jeśli znajduje się wewnątrz sieci korporacyjnej.

Spis treści

1 Wstęp

Wraz z dynamicznym rozwojem aplikacji internetowych oraz popularyzacją architektury rozproszonej, mechanizmy uwierzytelniania i autoryzacji stały się jednym z najbardziej krytycznych elementów systemów informatycznych. Zapewnienie bezpieczeństwa danych, przy jednoczesnym zachowaniu wysokiej wydajności i skalowalności, stanowi wyzwanie, z którym inżynierowie oprogramowania mierzą się od początków istnienia sieci WWW. We wcześniejszych fazach rozwoju aplikacji internetowych serwery generowały statyczne strony HTML, a potrzeba śledzenia tożsamości użytkownika była marginalna. Z czasem standardem stało się podejście oparte na sesjach, gdzie serwer tworzył w pamięci operacyjnej obiekt sesji, a do przeglądarki klienta wysyłał jedynie identyfikator zapisywany w pliku cookie. Choć bezpieczne, rozwiązanie to utrudniało skalowanie poziome w nowoczesnych architekturach mikroserwisowych.

Współczesne podejście do cyberbezpieczeństwa coraz częściej opiera się na paradygmacie *Zero Trust* (architekturze zerowego zaufania). Koncepcja ta zrywa z tradycyjnym modelem "zaufanej sieci wewnętrznej". W modelu *Zero Trust* zakłada się, że zagrożenie może pojawić się zewsząd, dlatego każde, nawet wewnętrzne żądanie dostępu do zasobów musi zostać rygorystycznie zweryfikowane. Oznacza to, że uwierzytelnienie przy wejściu do systemu nie jest wystarczające – niezbędna jest ciągła weryfikacja uprawnień (autoryzacja) przy każdym pojedynczym zapytaniu. Aby realizować ten postulat wydajnie, branża oprogramowania odeszła od obciążających serwery sesji na rzecz lekkich, kryptograficznie podpisanych poświadczeń.

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz implementacja autorskiego, wysoce zoptymalizowanego systemu autoryzacji w środowisku .NET, opartego o funkcjonalność tokenów. Stworzone rozwiązanie stanowi alternatywę dla standardu JWT, optymalizując narzut sieciowy oraz proces walidacji ról za pomocą operacji bitowych. Praca została podzielona na logiczne etapy, prowadząc czytelnika od przeglądu teorii i stosu technologicznego (Rozdział 2), poprzez projekt architektury i diagramy pozbawione szczegółów implementacyjnych (Rozdział 3), aż po konkretną realizację programistyczną (Rozdział 4). Dopełnieniem procesu inżynierskiego jest testowanie rozwiązania i analiza jego bezpieczeństwa w kontekście standardów branżowych, takich jak OWASP Top 10 (Rozdział 5), co ostatecznie prowadzi do podsumowania uzyskanych wyników.

2 Przegląd technologii i stan wiedzy

Aby w pełni zrozumieć architekturę proponowanego rozwiązania, konieczne jest omówienie fundamentów teoretycznych oraz technologii wykorzystywanych w nowoczesnych aplikacjach webowych. Rozdział ten wprowadza pojęcia protokołu HTTP, dogłębnie analizuje ewolucję mechanizmów utrzymania stanu, poddaje krytyce popularny standard JWT oraz definiuje stos technologiczny wybrany do implementacji autorskiego systemu.

2.1 Uwierzytelnianie a autoryzacja

W inżynierii bezpieczeństwa oprogramowania pojęcia uwierzytelniania i autoryzacji są często używane zamiennie, co stanowi poważny błąd terminologiczny. Zrozumienie różnicy między nimi jest kluczowe dla poprawnego modelowania systemów dostępu.

Uwierzytelnianie (ang. Authentication - AuthN) to proces weryfikacji tożsamości użytkownika lub systemu. Odpowiada na pytanie: "Kim jesteś?". W tradycyjnych systemach proces ten realizowany jest najczęściej poprzez podanie loginu i hasła. W rozwiązaniach bardziej zaawansowanych stosuje się biometrię lub uwierzytelnianie wieloskładnikowe (MFA).

Autoryzacja (ang. Authorization - AuthZ) to proces decyzyjny następujący zawsze po udanym uwierzytelnieniu. Odpowiada na pytanie: "Czy masz prawo to zrobić?". Nawet jeśli tożsamość użytkownika została poprawnie zweryfikowana, system autoryzacyjny musi sprawdzić, czy jego rola uprawnia go do dostępu do konkretnego zasobu (np. usunięcia rekordu z bazy danych). Niniejsza praca skupia się na optymalizacji obu tych procesów, ze szczególnym naciskiem na minimalizację kosztów obliczeniowych podczas ciągłej autoryzacji zapytań sieciowych.

2.2 Protokół HTTP i problem braku stanu

Protokół HTTP (ang. *Hypertext Transfer Protocol*), będący fundamentem komunikacji w sieci Web, z definicji jest protokołem bezstanowym (ang. *Stateless*). Oznacza to, że każde żądanie wysyłane przez klienta do serwera traktowane jest jako całkowicie niezależna operacja. Serwer na poziomie sieciowym nie przechowuje żadnych informacji o poprzednich interakcjach z danym klientem.

Z inżynierskiego punktu widzenia bezstanowość HTTP jest jego ogromną zaletą. Pozwala ona na szybkie obsługiwanie tysięcy jednoczesnych połączeń bez drastycznego przyrostu zużycia pamięci operacyjnej (RAM) serwera. Wymusza to jednak na twórcach oprogramowania aplikacyjnego budowę mechanizmów, które do każdego żądania dołączą dowód tożsamości użytkownika.

2.3 Ewolucja: Od sesji stanowych do bezstanowych tokenów

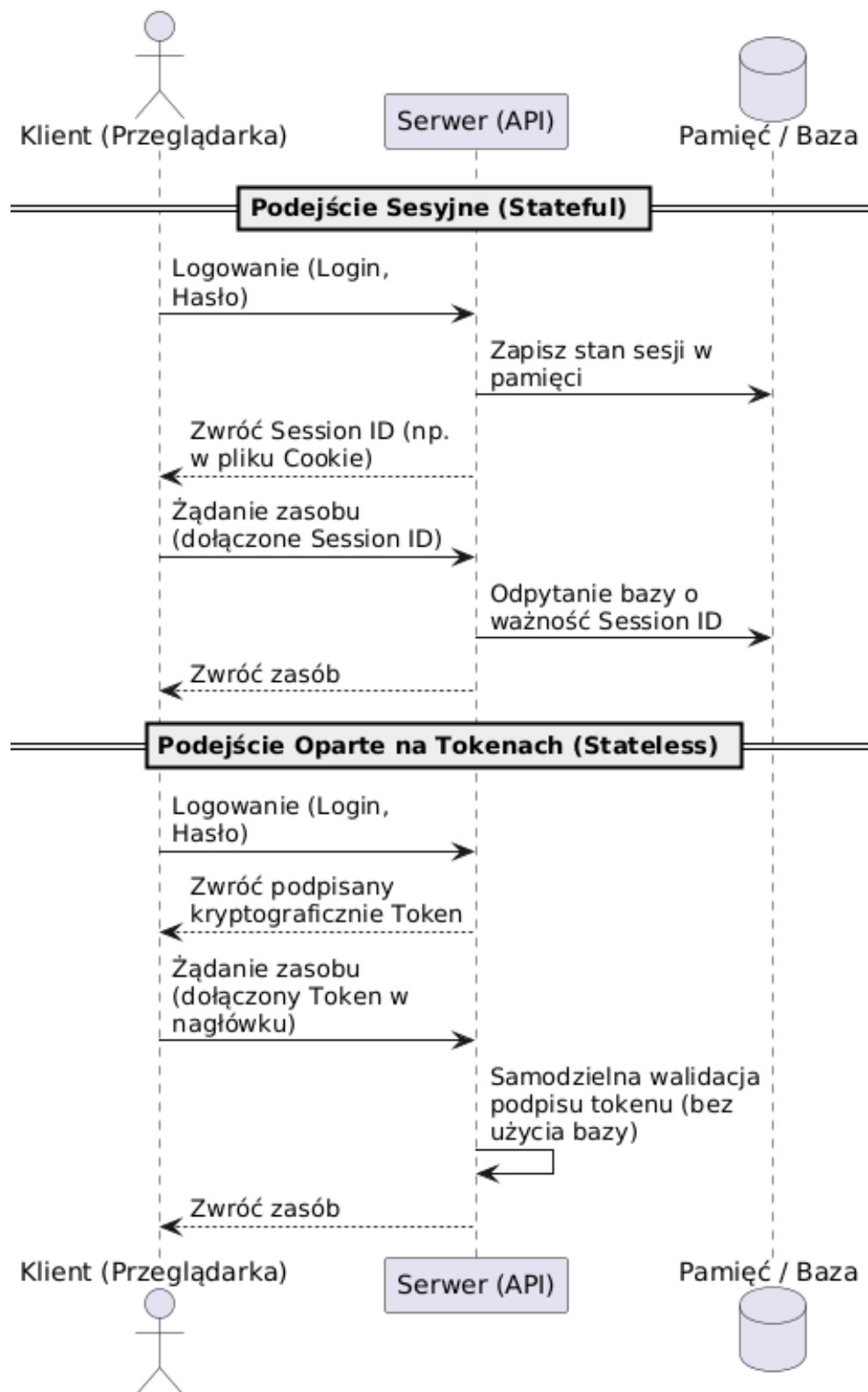
Początkowo standardem utrzymywania ciągłości pracy użytkownika było podejście oparte na sesjach (ang. *Session-based Authentication*). W tym modelu serwer, po zweryfikowaniu hasła, alokował w swojej pamięci operacyjnej obiekt sesji, a klientowi odsyłał jedynie krótki identyfikator (tzw. *Session ID*) za pomocą mechanizmu ciasteczek (ang. *Cookies*). Przeglądarka automatycznie dołączała ten identyfikator do kolejnych zapytań, a serwer za każdym

razem musiał przeszukiwać swoją pamięć lub bazę danych, aby upewnić się, że sesja jest nadal ważna.

Złotą erę tego podejścia zakończył rozwój architektury rozproszonej i chmury obliczeniowej. W środowiskach mikroserwisowych, gdzie ruch rozkładany jest na wiele niezależnych instancji serwerowych (skalowanie poziome), stanowe sesje stały się tzw. wąskim gardłem (ang. *bottleneck*). Jeśli pierwsze żądanie użytkownika trafiło na serwer A, który zapisał sesję, a kolejne na serwer B, uwierzytelnienie kończyło się błędem.

Rozwiązaniem tego problemu stały się tokeny. W modelu *Token-based Authentication* serwer podpisuje pakiet informacji reprezentujący tożsamość użytkownika, a następnie odsyła go do klienta. Klient przechowuje token i dołącza go do kolejnych żądań HTTP (najczęściej w nagłówku *Authorization*). Różnice w przepływie sterowania obu mechanizmów zobrazowano na Rysunku ??.

Jak widać na schemacie, podejście oparte na tokenach całkowicie odciąża bazę danych z procesu ciągłej autoryzacji – serwer samodzielnie weryfikuje podpis matematyczny.



Rysunek 2.1. Porównanie architektury stanowej (sesyjnej) z bezstanową (tokenową)

2.4 Anatomia standardu JSON Web Token (JWT)

Najpopularniejszym rynkowym formatem tokenu jest standard JWT (zdefiniowany w dokumencie RFC 7519). Klasyczny token JWT to długi ciąg znaków (często przekraczający 500 bajtów), który składa się z trzech części oddzielonych kropkami:

1. Nagłówek (Header): Definiuje typ tokenu oraz algorytm kryptograficzny użyty do podpisu. Zazwyczaj ma postać prostego obiektu JSON:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Ładunek (Payload): Miejsce przechowywania oświadczeń (ang. *claims*), czyli faktycznych danych o użytkowniku, takich jak jego identyfikator (**sub**), data wygaśnięcia (**exp**) czy posiadane uprawnienia. W standardzie JWT role przesyłane są zazwyczaj jako pełne ciągi tekstowe:

```
{
  "sub": "1234567890",
  "name": "Jan Kowalski",
  "roles": ["admin", "moderator", "user"],
  "exp": 1516239022
}
```

3. Podpis (Signature): Kryptograficzny skrót obliczony z zakodowanego nagłówka i ładunku przy użyciu klucza tajnego.

Krytyczna analiza tego standardu uwidacznia jego redundancję w zamkniętych środowiskach mikroserwisowych. Przesyłanie w każdym pojedynczym żądaniu nagłówka informującego o typie algorytmu jest zbędnym narzutem danych, ponieważ aplikacja odczytująca token posiada już tę wiedzę z własnej konfiguracji wewnętrznej. Ponadto, tekstowy zapis ról potęguje objętość ładunku. Wyeliminowanie tych nadmiarowych elementów stało się główną motywacją do stworzenia autorskiego formatu opisanego w niniejszej pracy.

2.5 Paradygmat Inversion of Control i mechanizm DI

Zastosowany stos technologiczny wymusza na programistach przestrzeganie dobrych praktyk projektowych. Jednym z najważniejszych paradygmatów zastosowanych w architekturze biblioteki jest odwrócenie sterowania (ang. *Inversion of Control* – IoC) realizowane poprzez wzorzec Wstrzykiwania Zależności (ang. *Dependency Injection* – DI).

Zasada ta jest bezpośrednio powiązana z literą "D" w akronimie SOLID (ang. *Dependency Inversion Principle*), która mówi, że moduły wysokopoziomowe nie powinny zależeć od modułów niskopoziomowych, a obie warstwy powinny opierać się na abstrakcjach. Zamiast bezpośrednio tworzyć instancje obiektów (np. połączeń do bazy danych) wewnątrz klas przy użyciu słowa kluczowego **new**, komponenty systemu deklarują swoje zależności poprzez interfejsy w konstruktorach.

W ekosystemie ASP.NET Core wbudowany kontener DI samodzielnie zarządza cyklem życia obiektów. Rozwiązanie to jest krytyczne dla autorskiego systemu autoryzacji, ponieważ zapewnia mu wymaganą hermetyzację, wysoką testowalność (łatwe wstrzykiwanie atrap – tzw. mocków) oraz modułowość pozwalającą na integrację z dowolnym środowiskiem sieciowym.

2.6 Środowisko uruchomieniowe i persystencja danych

Do realizacji projektu wybrano środowisko uruchomieniowe **.NET** oraz język **C#**. Platforma ta, rozwijana przez firmę Microsoft, stanowi obecnie jeden z najbardziej zaawansowanych i wydajnych ekosystemów do budowy wielowątkowych aplikacji serwerowych (Web API).

Istotnym elementem architektury jest warstwa persystencji danych. Interakcja z relacyjną bazą danych odbywa się za pośrednictwem technologii Entity Framework Core (EF Core). Jest to zaawansowane narzędzie klasy ORM (ang. *Object-Relational Mapping*), które rozwiązuje problem tzw. niedopasowania impedancji (ang. *impedance mismatch*) pomiędzy światem programowania obiektowego a tabelami bazy danych.

Wykorzystanie EF Core pozwala na operowanie na bazie bez konieczności pisania jawnych zapytań w języku SQL. Kodowanie odbywa się przy pomocy składni LINQ (ang. *Language Integrated Query*), a framework automatycznie tłumaczy te komendy na bezpieczny i zoptymalizowany pod dany silnik dialekt SQL. Użycie tego narzędzia automatycznie zabezpiecza autorski system autoryzacji przed jednym z najgroźniejszych ataków sieciowych – *SQL Injection* – ponieważ wszystkie wartości wstawiane do bazy są domyślnie traktowane jako parametryzowane dane wejściowe.

3 Projekt i architektura systemu

Architektura omawianego systemu została zoptymalizowana pod kątem redukcji narzutu sieciowego oraz błyskawicznej walidacji. W tym rozdziale zaprezentowano projekt mechanizmów wewnętrznych, sformalizowane modele matematyczne algorytmów oraz przepływy procesów. Zgodnie z dobrymi praktykami dokumentacji architektonicznej, celowo pominięto na tym etapie bezpośrednie implementacje kodowe, skupiając się na logice systemowej.

3.1 Koncepcja autorskiego tokenu

W przeciwieństwie do powszechnych standardów, autorski token został zaprojektowany z myślą o minimalizacji przesyłanych bajtów. Jest to ciąg znaków składający się zaledwie z dwóch segmentów: ładunku (B) zawierającego właściwe oświadczenia i dane użytkownika oraz sumy kontrolnej (S).

Proces powstawania tokenu T można sformalizować za pomocą następującego równania:

$$T = \text{Base64Url}(B) + "." + \text{Base64Url}(S)$$

gdzie:

- T – finalny ciąg znaków reprezentujący token autoryzacyjny, przesyłany w nagłówku zapytania HTTP.
- B – ładunek (ang. *Body/Payload*), będący tekstową reprezentacją zserializowanego obiektu JSON, przechowującego dane sesji (np. zdekodowaną maskę ról oraz czas wygaśnięcia).
- S – suma kontrolna (ang. *Signature*), będąca binarnym wynikiem operacji kryptograficznej zabezpieczającej ładunek.
- $\text{Base64Url}()$ – funkcja kodująca dane do bezpiecznego formatu tekstowego.

Kluczowym zabiegiem inżynierskim w tym równaniu jest użycie algorytmu **Base64Url** zamiast standardowego Base64. Standardowe kodowanie wprowadza znaki takie jak $+$ oraz $/$, które posiadają specjalne znaczenie w adresach URL i nagłówkach HTTP (mogą prowadzić do błędów parsowania). Base64Url zastępuje je odpowiednio znakami $-$ (myślnik) oraz $_$ (podkreślenie), a także usuwa znaki dopełnienia $=$, co gwarantuje pełną kompatybilność protokołu sieciowego.

3.2 Algorytm podpisu cyfrowego

Bezpieczeństwo systemu opiera się na zapewnieniu integralności i niezaprzeczalności emitowanych poświadczeń. Do generowania sumy kontrolnej (S) wykorzystano algorytm symetryczny HMAC (ang. *Keyed-Hash Message Authentication Code*) w oparciu o silną funkcję skrótu SHA-256.

Wybór kryptografii symetrycznej podyktowany jest niespotykaną wydajnością oraz specyfiką aplikacji monolitycznych, gdzie serwer wydający i weryfikujący to ten sam podmiot, posiadający bezpieczny dostęp do pamięci operacyjnej. Pełny proces generowania podpisu σ można wyrazić wzorem matematycznym:

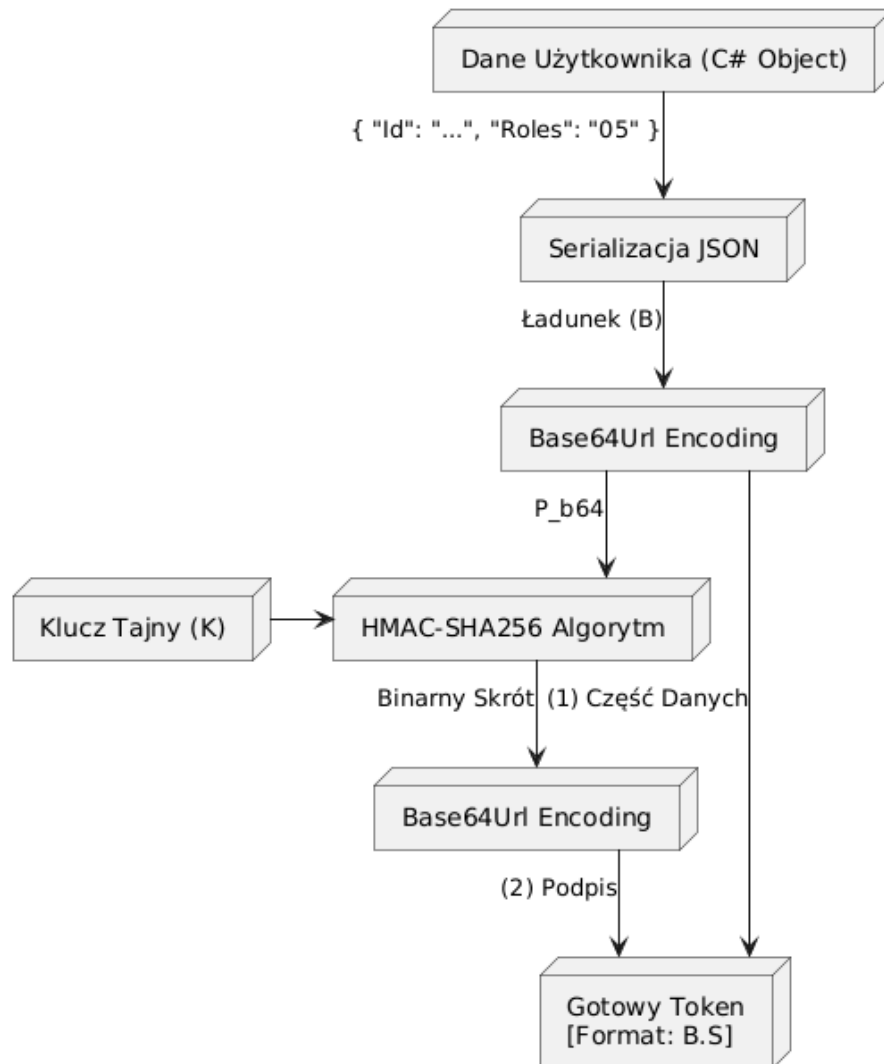
$$\sigma = \text{Base64Url}(\text{HMAC}_{\text{SHA256}}(K, P))$$

gdzie poszczególne zmienne oznaczają:

- σ – ostateczny, tekstowy podpis dołączany na końcu tokenu.

- K – klucz tajny (ang. *Secret Key*), będący ciągiem znaków o wysokiej entropii, zdefiniowanym w bezpiecznej konfiguracji środowiskowej serwera. Klucz ten nigdy nie jest przesyłany do klienta.
- P – ładunek wejściowy, czyli zakodowana już w Base64Url część danych ($P = \text{Base64Url}(B)$).
- $\text{HMAC}_{\text{SHA256}}$ – kryptograficzna funkcja mieszająca, która w bezpieczny sposób łączy klucz tajny z wiadomością, odporna na ataki typu *Length Extension Attack*.

Przepływ danych w procesie generowania tokenu, od momentu pobrania obiektu po uzyskanie końcowego ciągu znaków, zaprezentowano na Rysunku ??.



Rysunek 3.1. Architektura kryptograficzna generowania tokenu

3.3 Weryfikacja tokenu i integralność

Zrozumienie procesu weryfikacji jest kluczowe z perspektywy bezpieczeństwa. Weryfikacja nie polega na kryptograficznym "odszyfrowywaniu" sumy kontrolnej, lecz na procesie jej reprodukcji.

Kiedy serwer odbiera token z ządania HTTP, rozdziela go używając znaku kropki jako separatora. Następnie, korzystając z przechwyconego ładunku P oraz własnego klucza symetrycznego K , ponownie wykonuje funkcję $\text{HMAC}_{\text{SHA256}}$. Jeśli nowo wyliczona sygnatura σ' jest matematycznie tożsama z sygnaturą σ nadesłaną przez klienta ($\sigma' == \sigma$), serwer

nabiera pewności, że dane nie uległy manipulacji. Ewentualna zmiana choćby jednego bitu w ładunku (np. próba zmiany identyfikatora roli) spowoduje drastyczną zmianę wyliczonego skrótu zjawiskiem lawinowym (ang. *Avalanche Effect*), co skutkuje natychmiastowym odrzuceniem żądania HTTP z kodem 401 Unauthorized.

3.4 Zarządzanie rolami przy użyciu masek bitowych

Największą innowacją zaprojektowanego systemu jest całkowite odejście od tekstowej reprezentacji uprawnień. Model ten opiera się na matematycznej teorii zbiorów oraz systemie wag dwójkowych.

Niech R stanowi zbiór wszystkich dostępnych ról w systemie, a każda rola $r \in R$ posiada unikalny indeks liczbowy $ID_r \in \{0, 1, 2, \dots, n\}$. Zbiór ról przypisanych do konkretnego użytkownika oznaczmy jako $U \subseteq R$. Maska bitowa M dla danego użytkownika obliczana jest jako suma potęg liczby 2:

$$M = \sum_{r \in U} 2^{ID_r}$$

Dla przykładu, jeżeli system posiada role "user" ($ID = 0$) oraz "admin" ($ID = 2$), a użytkownik posiada obie z nich, jego maska wynosi:

$$M = 2^0 + 2^2 = 1 + 4 = 5$$

Liczba dziesiętna 5 jest następnie konwertowana do formatu szesnastkowego (05) i w takiej skompresowanej formie umieszczana w ładunku tokenu.

Kiedy użytkownik żąda dostępu do zasobu zabezpieczonego konkretną rolą r_{req} , serwer weryfikujący nie musi przeszukiwać list ani struktur tekstowych. Wykonuje on wyłącznie jedną, natywną dla procesora operację iloczynu bitowego (AND). Dostęp zostaje przyznany wtedy i tylko wtedy, gdy spełniony jest warunek:

$$(M \text{ AND } 2^{ID_{req}}) \neq 0$$

Działanie to omija kosztowną alokację pamięci, czyniąc proces weryfikacji niezwykle wydajnym.

3.5 Analiza złożoności obliczeniowej i pamięciowej

Sformalizowanie ról do postaci masek bitowych przynosi wymierne korzyści, co można udowodnić za pomocą notacji asymptotycznej (*Big O Notation*).

W standardowym podejściu (klasyczny JWT), role weryfikowane są poprzez iteracyjne przeszukiwanie tablicy ciągów znaków. Dla N ról posiadanych przez użytkownika oraz M ról wymaganych przez punkt końcowy, złożoność obliczeniowa najgorszego przypadku wynosi $O(N \cdot M)$. Ponadto, operacje na łańcuchach znaków (ang. *string comparisons*) dodatkowo obciążają pamięć sterty (ang. *Heap*) oraz wirtualną maszynę odśmiecającą pamięć (*Garbage Collector*).

W mechanizmie autorskim, po początkowym zdekodowaniu krótkiej maski szesnastkowej, proces walidacji sprowadza się do wywołania operatorów sprzętowych procesora. Złożoność obliczeniowa porównania bitowego, bez względu na wielkość słownika ról w systemie, jest stała i wynosi zawsze $O(1)$. Złożoność pamięciowa została zredukowana do alokacji pojedynczych zmiennych typów prostych (ang. *Value Types*).

3.6 Cykl życia sesji i rotacja tokenów odświeżających

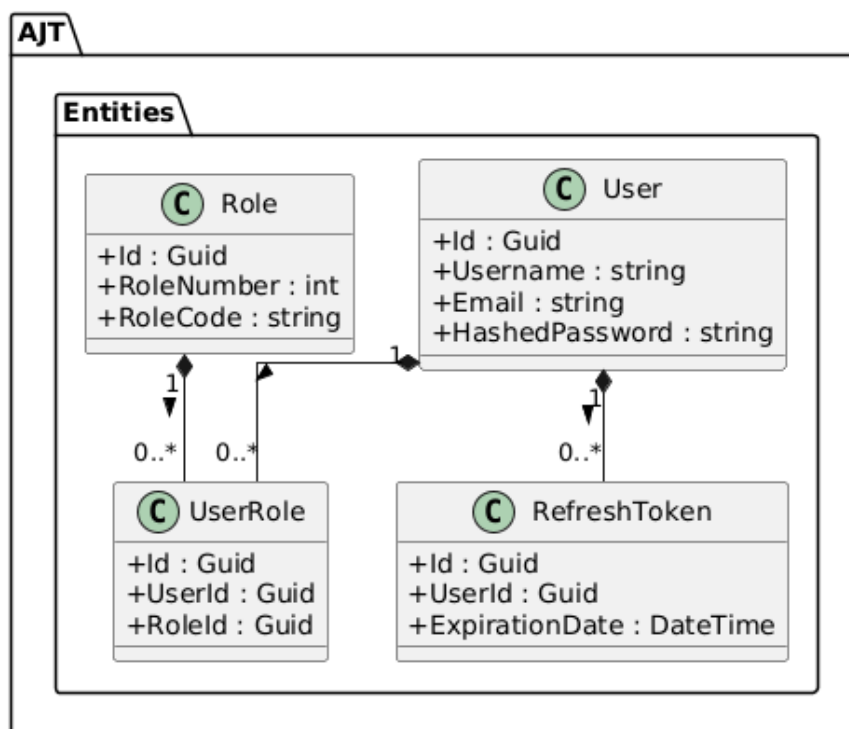
Krótki czas życia tokenu dostępowego (zazwyczaj od kilku do kilkunastu minut) stanowi fundament bezpieczeństwa w modelu bezstanowym. Ograniczenie to drastycznie minimalizuje okno czasowe, w którym przechwycony token mógłby posłużyć do wyrządzenia szkód w systemie.

Aby zapewnić ciągłość pracy bez wymuszania na użytkowniku ciągłego wpisywania hasła, wprowadzono długoterminowe tokeny odświeżające (ang. *Refresh Tokens*). Architektura zakłada wdrożenie mechanizmu rotacji (ang. *Refresh Token Rotation*).

Gdy token dostępowy traci ważność, aplikacja kliencka wysyła na dedykowany punkt końcowy token odświeżający. Jest to jedyny etap, w którym system wykonuje operację stanową (odpytuje relacyjną bazę danych). Po zweryfikowaniu kryptograficznym serwer upewnia się, że token istnieje w bazie i nie uległ przedawnieniu. Jeśli weryfikacja powiedzie się, stary token jest bezzwłocznie usuwany, a system emituje nową parę kluczy (nowy Access Token oraz nowy Refresh Token). Mechanizm ten gwarantuje, że skradziony token odświeżający będzie dla atakującego jednorazowy – gdy tylko prawowity użytkownik wygeneruje nową sesję, stary identyfikator zostanie bezpowrotnie unieważniony.

3.7 Diagram klas: Warstwa danych

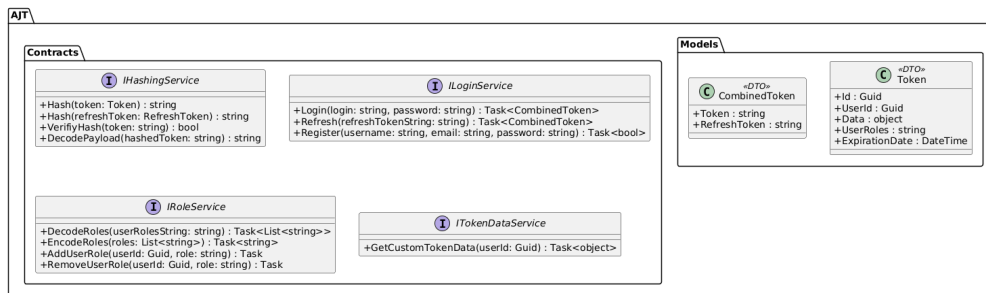
Z uwagi na zastosowanie wzorców inżynierskich system podzielono na logiczne pakiety, co zilustrowano za pomocą graficznej notacji UML. Rysunek ?? prezentuje encje bazodanowe mapujące obiekty C# na struktury relacyjne bazy. Widoczne jest powiązanie użytkownika z odpowiednimi tokenami odświeżającymi (relacja jeden-do-wielu), oraz przypisanie uprawnień poprzez tabelę asocjacyjną *UserRole*.



Rysunek 3.2. Diagram klas warstwy danych (Entities)

3.8 Diagram klas: Kontrakty i modele

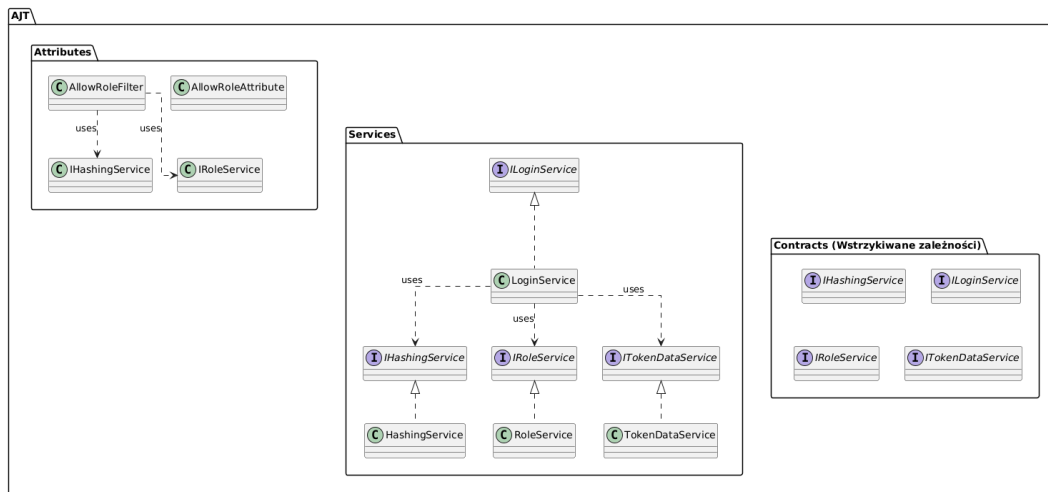
Rysunek ?? obrazuje obiekty transferu danych (DTO), takie jak model `Token` oraz jego zagnieźdzone elementy. Separacja abstrakcyjnych interfejsów (np. `IHashingService`) od ich implementacji fizycznych jest strukturalnym wymogiem obsługi wbudowanego kontenera Wstrzykiwania Zależności.



Rysunek 3.3. Diagram klas warstwy kontraktów i modeli

3.9 Diagram klas: Logika i punkty dostępu

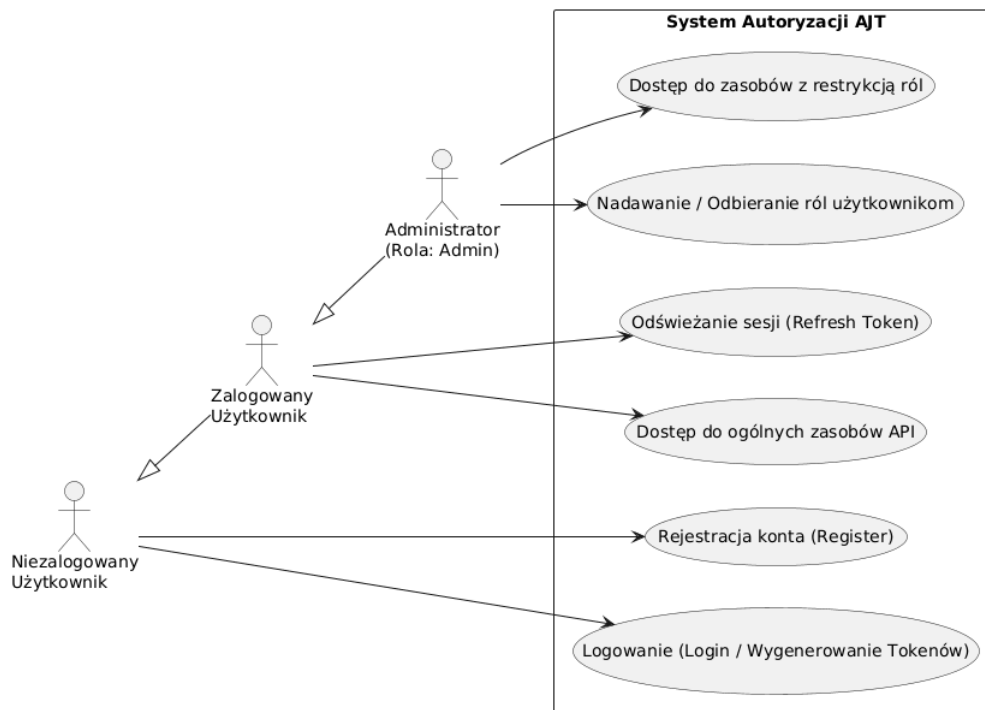
Ostatni schemat statyczny (Rysunek ??) skupia się na relacjach operacyjnych w warstwie biznesowej. Obrazuje zależność między wbudowanymi filtrami autoryzacyjnymi przechwytyjącymi żądania HTTP, a wstrzykiwanymi do nich implementacjami serwisów przetwarzającymi maski bitowe.



Rysunek 3.4. Diagram klas implementacji logiki biznesowej

3.10 Diagram przypadków użycia

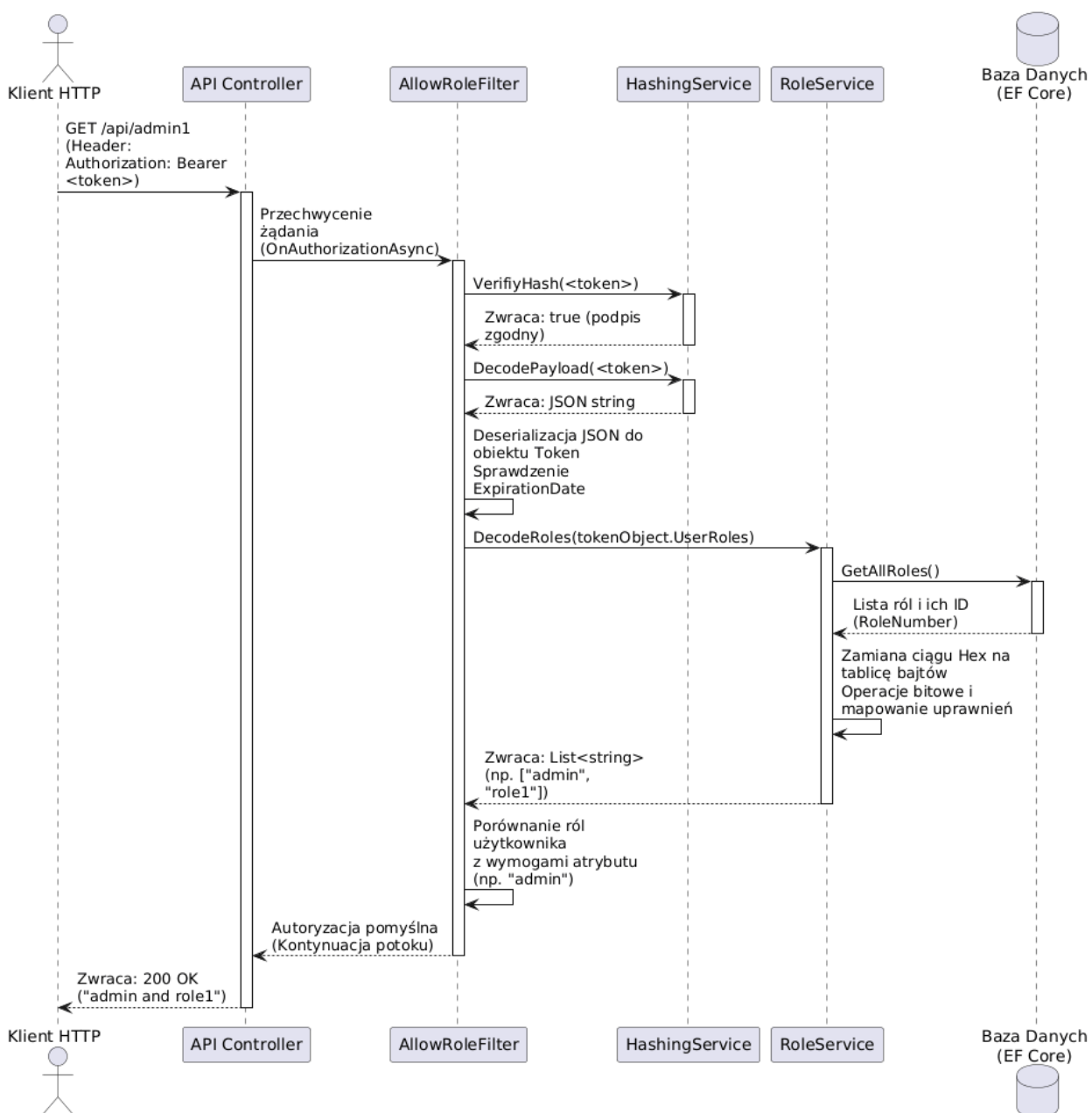
Zidentyfikowano trzech głównych aktorów wchodzących w interakcję z modulem (gość, użytkownik zalogowany, administrator). Jak zobrazowano na Rysunku ??, architektura opiera się na dziedziczeniu uprawnień, gdzie wyższy poziom dostępu wchłania kompetencje poziomu niższego, odblokowując zarazem administracyjne punkty dostępowe.



Rysunek 3.5. Diagram przypadków użycia systemu

3.11 Diagram sekwencji weryfikacji uprawnień

Sekwencyjny przepływ weryfikacji dostępu przedstawiono na Rysunku ?. Proces ten aktywuje się w momencie dotarcia żądania HTTP do serwera, tuż przed wywołaniem właściwej logiki domenowej. Przedstawiono w nim cykl życia walidacji: od weryfikacji kryptograficznej, przez dekodowanie matematycznej maski ról, aż po ostateczne przerwanie lub kontynuację potoku wykonawczego.



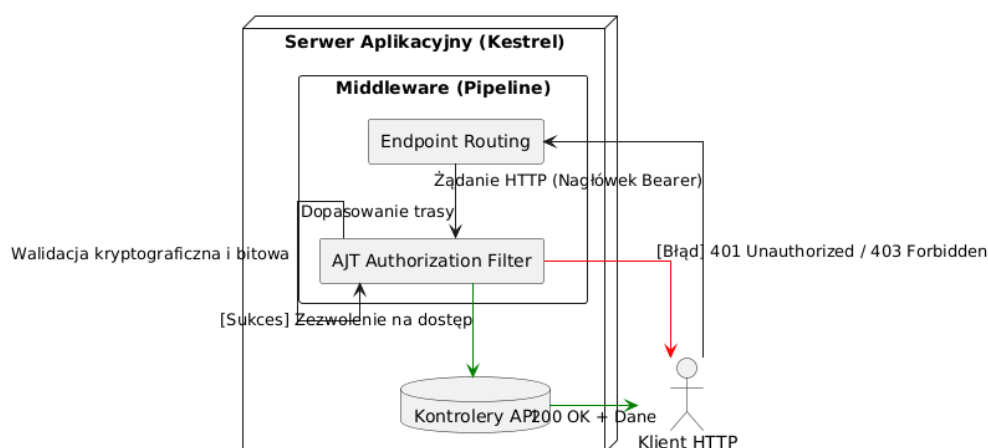
Rysunek 3.6. Diagram sekwencji weryfikacji uprawnień

4 Implementacja rozwiązania

Mając na uwadze przygotowany w poprzednim rozdziale teoretyczny model architektury, przystąpiono do właściwego programowania w języku C#. Zgodnie z wytycznymi czystego kodu (ang. *Clean Code*) oraz zasady rzemiosła oprogramowania (ang. *Software Craftsmanship*), w tej części pracy zaprezentowano wyłącznie wyselekcjonowane, kluczowe fragmenty kodu źródłowego. Zrezygnowano z przytaczania pełnych klas na rzecz wnikliwej analizy mechanizmów autoryzacyjnych.

4.1 Logika biznesowa i potok ASP.NET Core

Zanim użytkownik uzyska dostęp do chronionych zasobów, aplikacja musi przetworzyć jego zapytanie. Architektura platformy ASP.NET Core opiera się na potoku przetwarzania (ang. *Request Pipeline*), zbudowanym z oprogramowania pośredniczącego (*Middleware*) oraz filtrów. Nasz autorski system wstrzykuje swoją logikę decyzyjną dokładnie w ten potok, tuż przed przekazaniem sterowania do kontrolera biznesowego aplikacji, co zilustrowano na Rysunku ??.



Rysunek 4.1. Integracja filtra autoryzacyjnego z potokiem żądań ASP.NET Core

Zanim jednak filtr będzie mógł zwalidować żądanie (jak pokazano na schemacie), użytkownik musi najpierw wygenerować cyfrowe poświadczenie autentyczności, komunikując się z dedykowanym serwisem logowania.

4.2 Proces logowania i generowania tokenów

Serwis *LoginService* to główny punkt wejścia dla niezalogowanych użytkowników. Po poprawnej weryfikacji hasła system orkiestruje proces budowy i podpisywania poświadczeń. Użyto tu asynchronicznego dostępu do bazy danych, aby nie blokować głównego wątku serwera. Poniższy fragment ukazuje proces tworzenia zunifikowanego obiektu DTO (ang. *Data Transfer Object*) zawierającego parę tokenów.

```
public async Task<CombinedToken?> Login(string login, string password)
{
    // ... walidacja użytkownika i skrótu hasła ...
}
```

```

var userRoles = await userRoleRepo.GetUserRoles(user);

var token = new Token
{
    Id = Guid.NewGuid(),
    UserId = user.Id,
    Data = await _tokenDataService.GetCustomTokenData(user.Id),
    ExpirationDate = DateTime.Now.Add(_options.Value.TokenExpirationTime)
};

if (userRoles is not null)
    token.UserRoles = await _roleService.EncodeRoles(
        userRoles.Select(x => x.RoleCode).ToList());

// ... wygenerowanie i zapis obiektu RefreshToken w bazie ...

return new CombinedToken
{
    Token = _hashingService.Hash(token),
    RefreshToken = _hashingService.Hash(refreshToken)
};
}

```

Analizując powyższy kod, należy zwrócić uwagę na wykorzystanie wstrzykniętych zależności. Serwis `_roleService` przejmuje na siebie ciężar transformacji listy ról na format szesnastkowy (kompresja ról). Na samym końcu wywoływana jest metoda `Hash`, która zamienia struktury obiektowe C# na docelowe ciągi znaków (ciągi zabezpieczone HMAC-SHA256).

4.3 Implementacja odświeżania sesji i rotacji tokenów

Zgodnie z wymaganiami architektonicznymi, czas życia wydanego tokenu (`TokenExpirationTime`) jest krótki. Proces podtrzymywania sesji znajduje się wewnątrz metody `Refresh`. Przyjmuje ona wygasający token odświeżający i implementuje proces rotacji kluczy w sposób bezpieczny.

W pierwszej kolejności następuje weryfikacja kryptograficzna. Odrzuca ona zapytania ze zmodyfikowanym ładunkiem od razu, oszczędzając zasoby serwera bazy danych. Następnie system weryfikuje ważność tokenu w warstwie persystencji. Kluczowym momentem w poniższym kodzie jest wywołanie metody `DeleteRefreshToken`. Zużycie starego tokenu natychmiast po jego weryfikacji, a przed wydaniem nowej pary poświadczeń, stanowi podstawowe zabezpieczenie przed atakami typu powtórzenia (ang. *Replay Attack*).

```

public async Task<CombinedToken?> Refresh(string refreshTokenString)
{
    // 1. Szybka weryfikacja integralności kryptograficznej
    if (!_hashingService.VerifyHash(refreshTokenString))
        return null;

    var payloadJson = _hashingService.DecodePayload(refreshTokenString);
    var oldRefreshToken = JsonConvert.DeserializeObject<RefreshToken>(payloadJson);

    using var scope = _scopeFactory.CreateScope();
    var refreshRepo = scope.ServiceProvider.GetRequiredService<IRefreshTokenRepo>();

    // 2. Weryfikacja obecności i ważności tokenu w relacyjnej bazie danych

```

```

var dbToken = await refreshRepo.GetRefreshToken(oldRefreshToken.Id);
if (dbToken is null || dbToken.ExpirationDate < DateTime.Now)
    return null;

// 3. Rotacja - natychmiastowe usunięcie/zużycie starego tokenu
await refreshRepo.DeleteRefreshToken(dbToken.Id);

var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
var user = await userRepo.GetUserById(dbToken.UserId);

// 4. Wygenerowanie nowej pary (Access Token + nowy Refresh Token)
return await CreateCombinedTokenForUser(scope, user);
}

```

4.4 Dynamiczne rozszerzanie ładunku (Custom Payload)

Aby stworzona biblioteka mogła funkcjonować jako uniwersalne narzędzie typu *Plug and Play* w dowolnym projekcie, nie mogła ograniczać się wyłącznie do przechowywania ID użytkownika i ról. Zastosowano wzorzec strategii z wykorzystaniem delegatów `Func` zdefiniowanych w języku C#.

Poprzez konfigurację środowiskową, programista aplikacji nadrzędnej może wstrzyknąć funkcję zwrotną (ang. *Callback*). Funkcja ta jest wywoływana dynamicznie podczas każdego procesu logowania. Daje ona dostęp do `IServiceProvider`, co pozwala odpytać inne moduły systemu (np. zewnętrzny serwis HR o numer departamentu pracownika) i "dokleić" ten niestandardowy obiekt do właściwości `Data` w tokenie.

```

// Delegat wywoływany wstrzyknięty podczas konfiguracji biblioteki
public void AddCustomTokenDataFunc(Func<Guid, IServiceProvider, Task<object>> func)
{
    _customTokenDataFunc = func;
}

// Użycie delegata wewnątrz usługi budującej ładunek tokenu
public async Task<object> GetCustomTokenData(Guid userId)
{
    if (_options.Value.CustomTokenDataFunc != null)
        return await _options.Value.CustomTokenDataFunc(userId, _serviceProvider);

    return null; // Zwróć null, jeśli deweloper nie rozszerzył modelu
}

```

4.5 Automatyzacja zarządzania rolami (Refleksja)

Mechanizm operacji bitowych do weryfikacji ról jest wysoce zoptymalizowany, ale wymusza, by każda rola posiadała unikalny, sekwencyjny identyfikator numeryczny. Aby zdjąć z dewelopera obowiązek ręcznego uzupełniania bazy danych po dopisaniu nowej roli w kodzie, zaimplementowano tło asynchroniczne – `RoleBootstrapper`.

Klasa ta implementuje interfejs `IHostedService`, co oznacza, że uruchamia się przed otwarciem głównych portów nasłuchujących serwera. Wykorzystując potężny mechanizm refleksji przestrzeni `System.Reflection`, algorytm ładuje do pamięci wszystkie skompilowane biblioteki (ang. *Assemblies*), lokalizuje kontrolery i skanuje je pod kątem występowania atrybutów `[AllowRole]`.

```
protected override async Task ExecuteAsync(CancellationToken cancellationToken)
{
    // ... inicjalizacja dostępu do repozytorium przez IServiceProvider ...

    // Użycie refleksji do odnalezienia wszystkich kontrolerów API
    var controllers = AppDomain.CurrentDomain.GetAssemblies()
        .SelectMany(x => x.GetTypes())
        .Where(x => typeof(ControllerBase).IsAssignableFrom(x));

    var rolesFromAttributes = new HashSet<string>();

    foreach (var controller in controllers)
    {
        // Pobranie atrybutów z klas i metod bez ich uruchamiania
        var attributes = controller.GetCustomAttributes<AllowRoleAttribute>(true);
        // ... wyciągnięcie nazw ról z atrybutów i dodanie ich do zbioru (HashSet)
    }

    // ... porównanie zawartości zbioru ze stanem bazy danych.
    // Dopisanie brakujących ról wraz z wygenerowaniem sekwencyjnego numeru.
}
```

Dzięki takiemu architektonicznemu podejściu system ról uczy się i adaptuje całkowicie samoczynnie (tzw. mechanizm *Self-Discovery*).

4.6 Implementacja kryptografii i ochrona Timing Attack

Serwis `HashingService` realizuje logiczny proces powstawania skrótu HMAC. To właśnie w tym miejscu standardowy ciąg Base64 pozbawiany jest znaków kłopotliwych dla nawigacji HTTP.

Co niezwykle istotne, podczas implementacji weryfikacji podpisów w metodzie `VerifyHash` zidentyfikowano potencjalne zagrożenie związane z atakami czasowymi (ang. *Timing Attacks*). Standardowy operator porównania ciągów znaków (np. `==`) przerywa działanie natychmiast po napotkaniu pierwszej różnicy. Atakujący, analizując mikrosekundowe opóźnienia w odpowiedziach serwera przy przesyłaniu spreparowanych podpisów, mógłby krok po kroku odgadnąć cały skrót. Z tego powodu zaimplementowano stałoczasową weryfikację.

```
public bool VerifyHash(string token)
{
    // ... podział tokenu na ładunek i podpis ...
}
```

```

    string expectedSignature = CreateSignature(unsignedToken, _options.Value.Secret);

    var signatureBytes = Encoding.UTF8.GetBytes(signature);
    var expectedBytes = Encoding.UTF8.GetBytes(expectedSignature);

    // Zabezpieczenie: Stałoczasowe porównanie zapobiegające Timing Attack
    return CryptographicOperations.FixedTimeEquals(signatureBytes, expectedBytes);
}

private static string CreateSignature(string unsignedToken, string secret)
{
    var keyBytes = Encoding.UTF8.GetBytes(secret);
    var messageBytes = Encoding.UTF8.GetBytes(unsignedToken);

    using var hmac = new HMACSHA256(keyBytes);
    var hash = hmac.ComputeHash(messageBytes);

    return Base64UrlEncode(hash); // Własna funkcja modyfikująca standardowy Base64
}

```

4.7 Filtr autoryzacyjny potoku HTTP

Zwieńczeniem całego systemu jest przechwytywanie zapytań (zaprezentowane na schemacie ??). Rdzeniem decyzyjnym jest klasa `AllowRoleFilter`, będąca implementacją `IAsyncAuthorizationFilter`.

Filtr sprawdza warunki bezpieczeństwa. Jeśli algorytm `VerifyHash` zawiedzie (zmodyfikowany ładunek lub inna manipulacja), zwracany jest kod statusu 401 `Unauthorized` (brak uwierzytelnienia). Z kolei jeśli użytkownik ma poprawny token, ale algorytm bitowy `DecodeRoles` wykaże brak wymaganej roli dla danego zasobu, zwracany jest kod statusu 403 `Forbidden` (brak autoryzacji do konkretnego zasobu), co stanowi implementacyjną realizację zasad rozdzielności tych pojęć.

```

public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
{
    // ... sprawdzenie i wyciągnięcie nagłówka Bearer Token ...

    if (!_hashingService.VerifyHash(token))
    {
        context.HttpContext.Response.StatusCode = 401; // Unauthorized
        return;
    }

    // Dekodowanie identyfikatorów z formatu szesnastkowego z powrotem na nazwy
    var userRoles = await _roleService.DecodeRoles(tokenObject.UserRoles);

    // Weryfikacja zbioru uprawnień bitowych
    foreach (var requiredRole in _roles)
    {
        if (userRoles.Contains(requiredRole))
            return; // Zezwól na dostęp - przekaz żądanie do kontrolera
    }

    context.HttpContext.Response.StatusCode = 403; // Forbidden
}

```

5 Analiza bezpieczeństwa i testowanie

Istotnym etapem cyklu życia oprogramowania nastawionego na autoryzację jest empiryczna walidacja jego podatności. Rozdział ten skupia się na ocenie zaprojektowanego systemu przez pryzmat uznanego w branży standardu OWASP Top 10, szczegółowym omówieniu zautomatyzowanych testów bezpieczeństwa oraz nakreśleniu perspektyw dalszego rozwoju w aspektach, które wykraczały poza ramy obecnej iteracji prac.

5.1 Zgodność ze standardem OWASP Top 10

OWASP Top 10 to najpopularniejszy, globalny dokument uświadamiający zagrożenia bezpieczeństwa aplikacji internetowych. Biblioteka autoryzacyjna została zaprojektowana w oparciu o ideę *Security by Design*, co ogranicza ekspozycję na najczęstsze z klasyfikowanych podatności:

- **Broken Access Control (Naruszenie kontroli dostępu):** W modelu Zero Trust każda metoda API pozbawiona weryfikacji stanowi ryzyko eskalacji uprawnień. System eliminuje to zagrożenie poprzez sztywne wpięcie logiki decyzyjnej do potoku wykonawczego. Co więcej, rygorystyczne operacje oparte na matematycznych maskach bitowych nie pozwalają na manipulację ciągami tekstowymi ról (np. próbę nadpisania parametru `role="admin"`) przez nieuprzywilejowanych użytkowników.
- **Cryptographic Failures (Błędy kryptograficzne):** Poufność haseł została zapewniona poprzez wykorzystanie silnych skrótów kryptograficznych wraz z mechanizmem solenia (ang. *Salting*), co całkowicie uniemożliwia odzyskanie haseł tekstowych nawet w przypadku fizycznego wycieku bazy danych. Integralność samego tokenu opiera się o sprawdzony standard z rodziny SHA-256 i unikalny sekretny klucz.
- **Injection (Wstrzykiwanie kodu):** System wykorzystuje mapowanie obiektowo-relacyjne Entity Framework Core, które pod warstwą abstrakcji zawsze używa zapytań parametryzowanych. Architektura ta całkowicie neutralizuje zagrożenia związane z SQL Injection, uniemożliwiając wykonanie preparowanych zapytań modyfikujących strukturę relacyjną.
- **Identification and Authentication Failures (Błędy identyfikacji):** Implementacja rotacji tokenów odświeżających (opisana w poprzednich rozdziałach) oraz natychmiastowe usuwanie zużytych poświadczeń skutecznie chroni aplikację przed atakami typu *Session Fixation* oraz *Replay Attack* (atak polegający na przechwyceniu i ponownym wykorzystaniu starego żądania uwierzytelniającego).
- **Security Misconfiguration (Błędy konfiguracji zabezpieczeń):** Biblioteka wymaga na programiście dostarczenie klucza szyfrującego z poziomu konfiguracji zewnętrznej (poprzez wstrzyknięcie obiektu `AJTOptions`). Wymusza to odseparowanie kluczy od kodu źródłowego, chroniąc przed ich przypadkowym ujawnieniem w repozytoriach publicznych (np. GitHub).

5.2 Zautomatyzowane testy bezpieczeństwa (Security Unit Testing)

Najbardziej newralgicznym elementem zaprojektowanej architektury jest weryfikacja kryptograficzna podpisu, od której zależy ostateczne zaufanie do zawartości tokenu. Błąd na tym

etapie skutkowałby całkowitym przełamaniem systemu autoryzacji. Aby udowodnić nienaruszalność mechanizmu, kod poddano rygorystycznym testom jednostkowym z wykorzystaniem frameworka *xUnit* oraz biblioteki *FluentAssertions*.

Test 1: Odporność na wstrzykiwanie uprawnień (Tampering)

Pierwszy, najważniejszy scenariusz weryfikuje zjawisko wstrzykiwania uprawnień. Atakujący, znając format Base64Url, mógłby zdekodować pierwszą część tokenu, zmienić swoje uprawnienia (np. z "00" na "01" reprezentujące wyższą rolę), a następnie zakodować ją ponownie, pozostawiając stary podpis. Test dowodzi, że weryfikator natychmiast odrzuci tak zmodyfikowany ładunek.

[Fact]

```
public void VerifyHash_GivenTamperedToken_ShouldReturnFalse()
{
    // Arrange: Wygenerowanie poprawnego tokenu z najniższymi uprawnieniami
    var token = new Token { Id = Guid.NewGuid(), UserRoles = "00" };
    var hashedToken = _sut.Hash(token);

    // Act: Symulacja ataku poprzez modyfikację zawartości ładunku
    // bez przeliczania algorytmu HMAC (brak znajomości klucza tajnego).
    var tamperedToken = hashedToken.Replace("00", "01");

    var isValid = _sut.VerifyHash(tamperedToken);

    // Assert: Weryfikacja musi jednoznacznie odrzucić token
    isValid.Should().BeFalse();
}
```

Test 2: Izolacja kryptograficzna kluczy serwerowych

Kolejny scenariusz bezpieczeństwa zakłada próbę użycia tokenu podpisanego przez inny system (lub wygenerowanego przed rotacją głównego klucza serwera). Token poprawnie sformatowany, ale posiadający podpis wygenerowany innym kluczem tajnym, musi zostać uznany za nieautoryzowany. Zapobiega to wykorzystaniu fałszywych serwerów tożsamości do generowania tokenów dostępowych.

[Fact]

```
public void VerifyHash_SignedWithDifferentKey_ShouldReturnFalse()
{
    // Arrange: Generowanie tokenu przez Serwer A (Atakujący)
    var originalOptions = Options.Create(new AJTOptions { Secret = "Secret_A" });
    var attackerSut = new HashingService(originalOptions);
    var forgedToken = attackerSut.Hash(new Token { Id = Guid.NewGuid() });

    // Instancja głównego Serwera B z prawidłowym kluczem
    var validOptions = Options.Create(new AJTOptions { Secret = "Secret_B" });
    var validSut = new HashingService(validOptions);

    // Act: Próba autoryzacji fałszywym tokenem na właściwym serwerze
    var isValid = validSut.VerifyHash(forgedToken);

    // Assert: Token z obcym podpisem zostaje odrzucony
    isValid.Should().BeFalse();
}
```

Test 3: Odporność na uszkodzone struktury danych (Malformed Input)

Ataki polegające na przesyłaniu uszkodzonych ciągów znaków (np. całkowitym usunięciu podpisu lub zaburzeniu separatorów) często prowadzą do wywoływania nieobsłużonych

wyjątków serwera (ang. *Unhandled Exceptions*), co może być wektorem ataków typu DoS (ang. *Denial of Service*). Test wykorzystuje parametryzację ([Theory]), aby upewnić się, że funkcja weryfikująca "kulturalnie" odrzuca takie dane bez awarii aplikacji.

```
[Theory]
[InlineData("TylkoJedenSegmentBezKropki")]
[InlineData("Segment1.Segment2.Segment3.ZaDuzoKropek")]
[InlineData(".TylkoPodpis")]
[InlineData("TylkoLadunek.")]
[InlineData("")]
public void VerifyHash_GivenMalformedFormat_ShouldReturnFalse(string malformedInput)
{
    // Act
    var isValid = _sut.VerifyHash(malformedInput);

    // Assert: Wszystkie niepoprawne formaty dają wynik negatywny
    // Serwer nie ulega awarii (brak wyjątków NullReferenceException itp.)
    isValid.Should().BeFalse();
}
```

5.3 Ograniczenia systemu i obszary dalszego rozwoju

Świadomość ograniczeń wyprodukowanego kodu stanowi miarę inżynierskiej dojrzałości projektu. Choć system spełnia założenia optymalizacyjne w zakresie objętości danych, jego implementacja pozostawia pole do rozbudowy w przyszłych iteracjach badawczych.

Czarna lista tokenów (Blacklisting): W pierwszej kolejności nie zaimplementowano tzw. czarnej listy tokenów. Ponieważ serwer z założenia nie odpytuje bazy danych przy walidacji *Access Tokenu*, w przypadku jawnego wylogowania lub przejęcia poświadczeń z przeglądarki użytkownika, token pozostaje ważny aż do czasu naturalnego wygaśnięcia. Wdrożenie zewnętrznej, błyskawicznej bazy pamięciowej (np. Redis), która przechowywałaby identyfikatory unieważnionych tokenów z przypisanym czasem życia (ang. *Time-To-Live*) dopasowanym do ich wygaśnięcia, skutecznie załatałoby tę lukę bez drastycznego spadku wydajności.

Kryptografia asymetryczna: Brak implementacji kryptografii asymetrycznej ogranicza wdrożenie systemu do prostszych środowisk. Obecny symetryczny algorytm HMAC-SHA256 sprawdza się w przypadku monolitycznego API, lecz w wysoce rozproszonej architekturze wymagane byłoby współdzielenie tego samego klucza tajnego między wszystkimi jednostkami infrastruktury, co stanowi znaczne ryzyko kradzieży. Implementacja wsparcia dla algorytmów z rodziny RSA lub ECDSA (gdzie centralny serwer autoryzacyjny używa klucza prywatnego do podpisu, a mikroserwisy używają ogólnodostępnego klucza publicznego do weryfikacji) stanowiłaby krytyczne rozszerzenie możliwości biblioteki.

Ochrona przed atakami Brute-Force: Autorski system nie posiada wbudowanych mechanizmów limitowania zapytań sieciowych. Konsekwencją tego jest teoretyczna podatność punktu końcowego realizującego funkcję logowania na ataki siłowe (ang. *Brute-Force*) lub ataki słownikowe (ang. *Credential Stuffing*). W przyszłości biblioteka powinna zostać zintegrowana z natywnym modułem *ASP.NET Core Rate Limiting*, aby automatycznie blokować numery IP generujące anomalną liczbę błędnych prób uwierzytelnienia.

6 Podsumowanie

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie autorskiego, zoptymalizowanego systemu uwierzytelniania i autoryzacji dla aplikacji opartych o środowisko ASP.NET Core. Poprzez analizę powszechnie stosowanych technologii sieciowych oraz wymagań współczesnych norm bezpieczeństwa, udowodniono skuteczność paradygmatu *Zero Trust* wspomaganego szybkimi, bezstanowymi tokenami.

Opracowana biblioteka pomyślnie rozwiązuje problem strukturalnego narzutu przesyłanych danych obecnego w popularnym standardzie JWT. Eliminacja nieużywanych metadanych oraz innowacyjne, oparte na konwersji bitowej podejście do interpretowania złożonych struktur ról użytkowników zaowocowały miniaturyzacją autoryzacyjnych żądań sieciowych. Wykorzystanie warstwy abstrakcji programistycznej, zgodnej z wzorcami wstrzykiwania zależności, umożliwiło zamknięcie całej złożoności w zgrabnym, hermetycznym module o minimalnym progu wejścia (ang. *low entry barrier*) dla przyszłych programistów.

Przeprowadzona analiza testowa w oparciu o globalne wytyczne OWASP potwierdziła, że kompresja algorytmów nie wpłynęła na obniżenie parametrów zabezpieczeń kryptograficznych. Choć uwypuklono obszary do potencjalnej kontynuacji badań technologicznych – takie jak implementacja natywnych rozwiązań pamięci podręcznej dla autoryzacji czy kluczy asymetrycznych – wykreowane rozwiązanie spełnia zadane wymagania inżynierskie i stanowi w pełni funkcjonalną alternatywę autoryzacyjną dla systemów o klasycznym i mikroserwisowym profilu architektonicznym.

Wyrażam zgodę na udostępnienie mojej pracy w czytelniach Biblioteki SGGW w tym
w Archiwum Prac Dyplomowych SGGW.

.....
(czytelny podpis autora pracy)

